



Smart Classroom Attendance System

using Face Recognition and Liveness Detection

AI · Computer Vision · Deep Learning · Web & Mobile Application · Database

A detailed guide to the complete system

What the system does and why, how every AI component works, the architecture, database, modules, web and mobile app, security, evaluation on real data, installation, and a ready-to-use presentation and viva guide.

Student(s): _____

Registration no.: _____

Supervisor: _____

Department: Computer Science

Session: _____

How to use these notes

These notes explain the complete **Smart Classroom Attendance System**, from the problem it solves to the mathematics inside the AI models, so that you can present the project, answer viva questions and write your final report. Every number quoted in the results chapters was measured on the actual system. Chapters 1–3 give the big picture. Chapters 4–6 explain the AI (the most important part for a viva). Chapters 7–12 describe the software engineering. Chapters 13–15 cover results, installation and limitations. Chapter 16 is a ready-made presentation plan with a demo script and likely viva questions.

Reading tip

If you only have one hour before your presentation, read Chapter 1 (overview), Chapter 4 (AI pipeline), Chapter 13 (results) and Chapter 16 (presentation guide).

Contents

1. Introduction and Project Overview	5
1.1 The problem	5
1.2 The solution in one paragraph	5
1.3 Objectives	6
1.4 Scope	6
1.5 Why this is an AI project	6
2. Features and Users	8
2.1 Feature list	8
2.2 User roles	8
2.3 A typical day with the system	8
3. System Architecture and Technology	10
3.1 Layered architecture	10
3.2 Technology stack	11
3.3 Project structure	11
4. The AI Pipeline: Detection and Recognition	12
4.1 Background: convolutional neural networks	12
4.2 Step 1 — Face detection with YuNet	12
4.3 Improvement: multi-scale detection (found on real data)	12
4.4 Step 2 — Face alignment	13
4.5 Step 3 — Face embedding with SFace	13
4.6 Step 4 — Matching with cosine similarity	13
4.7 Step 5 — Tracking faces between frames	14
4.8 Step 6 — Multi-frame voting	14

4.9 Enrolment: how students are registered	15
5. Liveness Detection and Anti-Spoofing	16
5.1 Method 1 — Anti-spoofing CNN (passive)	16
5.2 Method 2 — Head-movement test (own algorithm)	16
5.3 Liveness modes	17
6. Decision Logic: From Recognition to Attendance	18
6.1 States of a face on the live screen	18
6.2 Attendance status rules	18
6.3 Duplicate prevention	18
6.4 Other rules	19
7. Database Design	20
8. Module-by-Module Walkthrough	22
8.1 Login and roles	22
8.2 Student management and bulk import	22
8.3 Courses and enrolment	22
8.4 Sessions and live attendance	22
8.5 Analytics	23
8.6 Reports	24
8.7 Administration	24
9. Web Application and Mobile App	25
9.1 One code base, two kinds of app	25
9.2 Making phones trust the server	27
9.3 Verified behaviour	27
10. Security, Privacy and Ethics	28
11. Installation and Running the System	29
11.1 Requirements	29
11.2 First start (Windows)	29
11.3 Useful commands	29
11.4 Troubleshooting	29
12. Testing	31
13. Evaluation and Results	32
13.1 Metrics used	32
13.2 Face detection	32
13.3 Face recognition	33
13.4 Liveness	33

13.5 End-to-end system test on real data	34
13.6 Speed	34
14. Problems Found and Solved	35
15. Limitations and Future Work	36
15.1 Limitations	36
15.2 Future work	36
16. Presentation and Viva Guide	37
16.1 Suggested slides (15–18 minutes)	37
16.2 Live demo script	37
16.3 Likely viva questions and answers	38
Appendix A: Glossary	40
Appendix B: Configuration and API Reference	41
B.1 Main settings	41
B.2 Main endpoints	41

1. Introduction and Project Overview

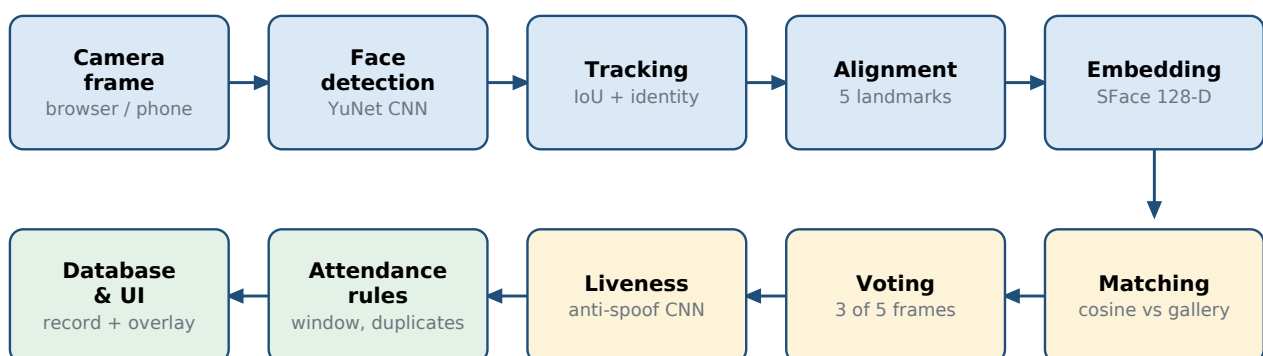
1.1 The problem

Taking attendance by roll call wastes 5–10 minutes of every lecture. In a semester with 30 lectures per course that is several hours of teaching time lost per course. Manual registers can also be wrong (a friend answers for an absent student, called proxy attendance). They are hard to analyse and they are usually typed into a computer later by hand. Card systems can be shared, and fingerprint scanners need special hardware, cause queues and require physical contact.

A face is something every student always carries and cannot lend to a friend. A normal camera, which every laptop and phone already has, can capture it without contact. The difficulty is that recognising faces reliably, in a real classroom with many students, varying light and people trying to cheat with photos, is a hard **Artificial Intelligence** problem.

1.2 The solution in one paragraph

The system automatically marks attendance by recognising students' faces through a camera. It detects every face in the camera image, turns each face into a numerical fingerprint (an embedding) with a deep neural network, and compares it with the registered students. It then checks that the face belongs to a live person and not a printed photo or a phone screen, and records the student as **Present**, **Late** or **Absent** according to configurable rules, exactly once per class. Everything is stored in a relational database and presented through a web application and an installable mobile app. The apps provide dashboards, analytics, low-attendance alerts and Excel/PDF/CSV reports for administrators, teachers and students.



Unknown face → red box 'Unknown', never marked

Photo / screen → pink box 'Spoof suspected', logged in the audit trail

Registered + live + enrolled → green box, marked Present / Late (once per session)

Figure 1: The end-to-end flow from camera frame to attendance record

1.3 Objectives

1. Automatically identify registered students from a live camera feed, including several students in one frame.
2. Reject unregistered persons (“Unknown”) so that attendance is never given to the wrong person.
3. Detect presentation attacks (printed photos, phone or laptop screens) with a liveness / anti-spoofing component.
4. Record attendance with date, time and status (Present / Late / Absent / Excused / Leave) and prevent duplicates.
5. Manage students, courses, teachers, enrolments and class sessions in a proper relational database.
6. Provide dashboards, analytics and exportable reports, and warn when a student's attendance falls below a threshold.
7. Work on ordinary hardware (a laptop with a webcam, or a phone) without a GPU or special devices.
8. Measure the accuracy of the AI honestly with standard metrics (FAR, FRR, ROC AUC, APCER) instead of just claiming that it works.

1.4 Scope

In scope	Out of scope
Face detection, recognition and liveness for classroom attendance	Training new neural networks from scratch
Web app for PC browsers and installable mobile app (PWA)	Publishing on Google Play / App Store (possible future work)
Students, courses, sessions, rules, reports, analytics, alerts	Integration with a university ERP / LMS
Single server for a department (SQLite or PostgreSQL)	Cloud multi-campus deployment
Evaluation scripts and measurements on real face data	Large-scale study with hundreds of real students

Table 1: Scope of the project

1.5 Why this is an AI project

The core of the system consists of three pre-trained deep-learning models (convolutional neural networks) and several AI decision techniques around them. Table 2 separates the AI components from the conventional software components. The project does **not** train its own networks, which is standard practice. Training a face recogniser needs millions of labelled faces and weeks of GPU time. The AI work in this project is choosing suitable models, integrating them into a real-time pipeline, discovering and fixing their failure cases on real data, designing the liveness method, choosing thresholds from measured error rates, and evaluating the result.

Component	Technique	AI?
Face detection	YuNet convolutional neural network, multi-scale	Yes (deep learning)
Face recognition	SFace deep embedding network (hypersphere margin loss, ArcFace family) + cosine similarity	Yes (deep learning)
Anti-spoofing	MiniFASNet CNN trained on CelebA-Spoof (live / print / replay)	Yes (deep learning)
Head-movement liveness	Own geometric algorithm on facial landmarks (affine invariance)	Yes (computer vision)
Tracking & voting	IoU tracker, identity-change detection, 3-of-5 majority vote	Yes (decision logic)
Attendance rules, database, reports, UI	Rule-based logic, SQL, web development	No (software engineering)

Table 2: AI and non-AI components

2. Features and Users

2.1 Feature list

Module	Features
1. User & student management	Student registration (ID, roll no., name, department, semester, section, e-mail, phone, consent); bulk import from Excel/CSV; courses; enrolments; teacher and student accounts
2. Face enrolment	Webcam capture (auto-capture 5), photo upload, bulk ZIP import; dominant-face selection; same-person check; duplicate-identity check; encrypted templates
3. AI face recognition	Multi-face, multi-scale detection; alignment; 128-D embeddings; matching with threshold and margin; unknown detection; tracking; voting
4. Attendance management	Class sessions; Present / Late / Absent windows; enrolment check; duplicate prevention; automatic absentees on close; manual override (Excused, Leave)
5. Liveness / anti-spoofing	Anti-spoofing CNN (passive); head-movement test (active); blur check; three configurable modes; spoof attempts logged
6. Analytics & reporting	Dashboard; daily / weekly / monthly trends; course-wise rates; best and worst students; absences by weekday; daily, monthly and session reports in Excel, PDF and CSV
7. Administration & security	Roles; system settings; login lock-out; password change; AI audit log; backups; HTTPS; data deletion
Mobile app	Installable PWA for Android and iPhone; front/back camera; offline screen; QR-code setup page

Table 3: Features grouped by module

2.2 User roles

Role	Can do
Administrator	Everything: students and faces, courses, teachers, users, system settings, all attendance, reports, audit log
Teacher	Own courses only: create and run sessions, live camera attendance, corrections, analytics, reports, alerts
Student	Own attendance only: percentage per course, history, current warnings (web or mobile app)

Table 4: Role-based access

2.3 A typical day with the system

- Once per semester (admin):** import the class list, record consent, register faces (about 20 seconds per student with the webcam, or a ZIP of phone photos).
- Before a lecture (teacher):** open the app, choose Attendance → Create session → Create & start.
- During the first minutes:** the phone or laptop camera watches the room. Each student looks at the camera for 2–3 seconds and gets a green box: “Amina Bibi – Present 09:03”.

4. **After the lecture:** the teacher clicks Close session. Everyone not seen is marked Absent automatically, and warnings are created for students below 75%.
5. **Any time:** teachers and admins view analytics and export reports; students check their own attendance on their phones.

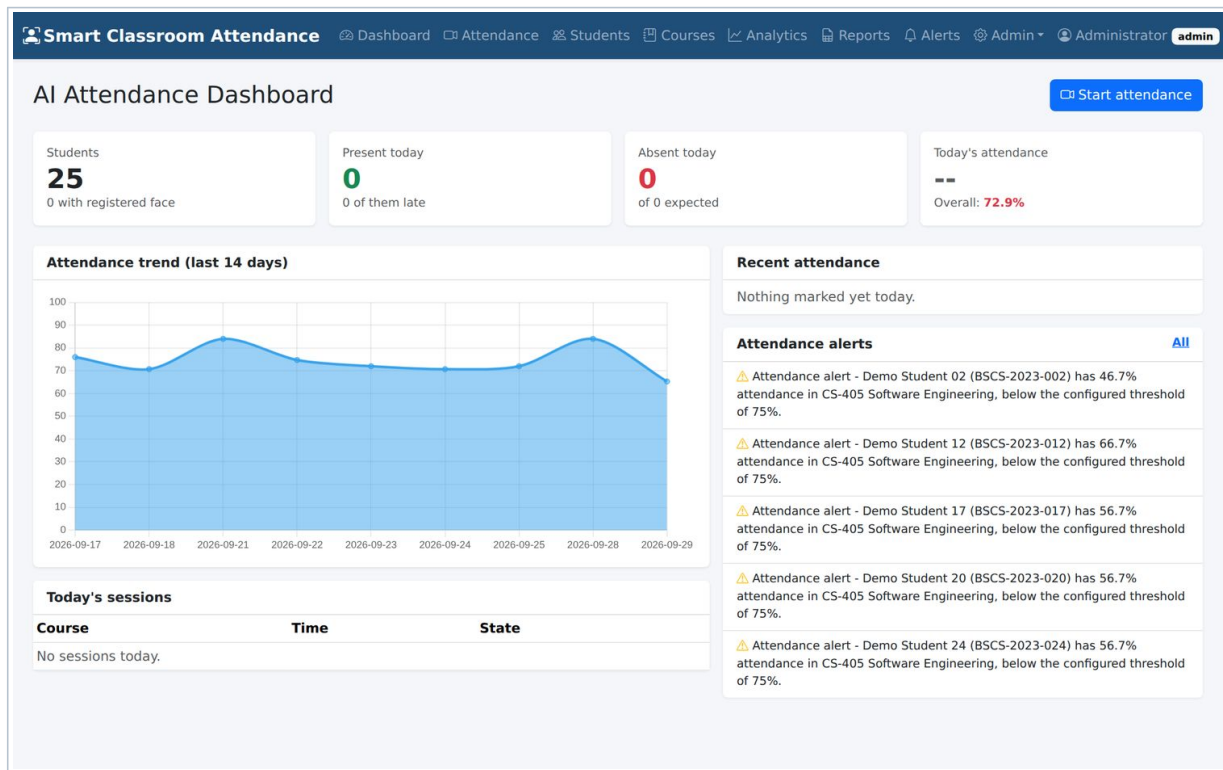


Figure 2: Web dashboard: key figures, trend chart, today's sessions and current attendance alerts

3. System Architecture and Technology

3.1 Layered architecture

The system follows a classic **layered architecture**. Each layer only uses the layer below it, which keeps the AI code independent from the web code and makes each part testable on its own.

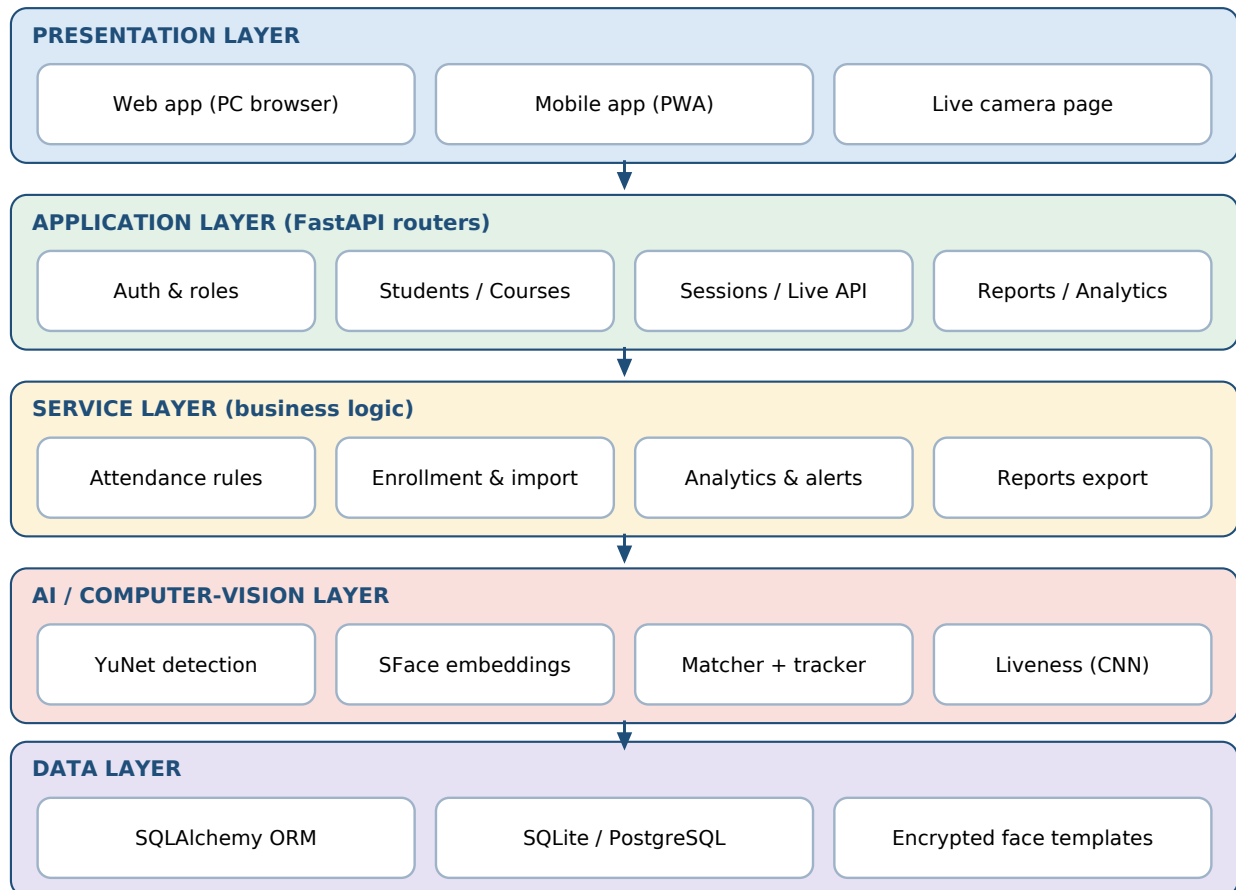


Figure 3: Layered architecture of the system

Layer	Responsibility	Main files
Presentation	HTML pages, live camera JavaScript, PWA (manifest, service worker)	app/templates, app/static
Application	HTTP endpoints, login sessions, role checks, input validation	app/routers/*.py, app/deps.py
Service	Attendance rules, enrolment, import, analytics, reports, alerts, settings	app/services/*.py
AI / Vision	Detection, embedding, anti-spoofing, matching, tracking, liveness	app/vision/*.py
Data	Tables, relations, constraints, encryption of templates	app/models.py, app/database.py, app/security.py

Table 5: Responsibilities of each layer

3.2 Technology stack

Area	Technology	Why it was chosen
Language	Python 3.10-3.14	The standard language for AI and computer vision; one language for AI and backend
Computer vision	OpenCV 4/5 (DNN module), NumPy	Runs ONNX neural networks fast on a normal CPU without PyTorch or a GPU
AI models	YuNet, SFace, MiniFASNet (ONNX)	Small, accurate, open models; 40 MB in total
Web framework	FastAPI + Uvicorn	Modern, fast Python web framework with validation and automatic API docs
Database	SQLAlchemy 2 ORM; SQLite (default) or PostgreSQL	SQLite needs no installation; PostgreSQL for a larger deployment
Frontend	Jinja2 templates, Bootstrap 5, Chart.js	Responsive pages that work on PC and phone; bundled so it works offline
Mobile	Progressive Web App (manifest + service worker)	One code base installs on Android and iPhone without an app store
Security	cryptography (Fernet), PBKDF2, HTTPS	Encrypted face templates, safe passwords, secure camera access
Reports	openpyxl, ReportLab, csv; segno (QR codes)	Excel, PDF and CSV exports; QR code for phone setup
Testing	pytest, Playwright (browser tests)	46 automated tests plus a real-data end-to-end test

Table 6: Technology stack

3.3 Project structure

run.py, start.bat, start-mobile.bat, start.sh launchers (HTTP locally, HTTPS for phones)
app/

main.py	FastAPI application, login handling, first admin account
config.py	all settings (environment variables / .env)
models.py	database tables (SQLAlchemy)
security.py	password hashing, encryption of face templates
vision/	AI: backends.py (YuNet, SFace, anti-spoof CNN), matcher.py, pipeline.py (tracking + voting), liveness.py
services/	attendance.py, faces.py, importer.py, analytics.py, reports.py, notifications.py, app_settings.py
routers/	auth, dashboard, students, imports, courses, sessions, analytics, reports, admin, mobile
templates/ static/	web pages, JavaScript (camera, live overlay), PWA files, icons
scripts/	download_models, import_data, evaluate, evaluate_liveness, simulate_photo_attack, webcam_attendance, backup, seed_demo
tests/	46 automated tests + real-data end-to-end test
docs/	architecture, evaluation, deployment, privacy & security

4. The AI Pipeline: Detection and Recognition

This chapter explains the heart of the system. The live page sends a camera frame (a JPEG image) to the server about twice per second. For every frame the server runs the steps in Figure 1. Each step is explained below with the reason for every design decision.

4.1 Background: convolutional neural networks

A **convolutional neural network (CNN)** is a deep-learning model designed for images. Its early layers learn to detect simple patterns such as edges and corners; deeper layers combine them into eyes, noses and whole faces. A CNN learns these patterns itself from many labelled examples during training. Using the trained network on new images is called inference, which is what this system does. All three networks are stored in the **ONNX** format (Open Neural Network Exchange) and run by OpenCV's DNN module on the CPU.

4.2 Step 1 — Face detection with YuNet

Face detection answers the question “where are the faces?”. YuNet is a very small CNN (about 230 KB, roughly 75 000 parameters) designed for real-time detection. For every face it outputs a bounding box (x, y, width, height), a confidence score between 0 and 1, and **five landmarks**: both eye centres, the nose tip and both mouth corners. It finds all faces in one pass, so a whole classroom can be processed per frame.

Setting	Value	Meaning
Score threshold	0.85	Detections with lower confidence are ignored (avoids false faces)
Minimum face size	40 px	Smaller faces are too small to recognise reliably
NMS threshold	0.3	Non-maximum suppression removes duplicate boxes of the same face
Scales	640 px and full resolution (≤ 1920 px)	Multi-scale detection, explained in §4.3

Table 7: Face detection settings

4.3 Improvement: multi-scale detection (found on real data)

When the system was tested on real photographs, **28% of the photos had no face detected**. Investigation showed why: YuNet is trained mostly on small and medium faces, and its confidence falls to 0.53–0.83 for faces wider than about 450 pixels (phone photos, or a student sitting close to the webcam). The same photos, downsampled, scored 0.93–0.96. The fix runs the detector twice: once on a copy reduced to 640 px (for large faces) and once at full resolution up to 1920 px (for small faces at the back of a room). The two sets of boxes are mapped back to the original coordinates and

merged with non-maximum suppression. Detection on the real photos rose from **72% to 100%**.

Viva point

This is a good example of engineering an AI system: a pre-trained model has limits, and testing on real data revealed a failure that you can explain, measure and fix.

4.4 Step 2 — Face alignment

People tilt and turn their heads, so the face is first **aligned**. Using the five landmarks, a similarity transform (rotation, scale and translation) maps the eyes, nose and mouth onto fixed positions of a standard 112×112 pixel face. After alignment every face looks “straight”. This makes recognition much more accurate because the network always sees the eyes in the same place.

4.5 Step 3 — Face embedding with SFace

Face recognition answers “who is this?”. The aligned face is passed through **SFace**, a deep CNN (based on MobileFaceNet) that outputs **128 numbers**, the face embedding. The network was trained with the **SFace loss (sigmoid-constrained hypersphere loss)**, a margin-based loss from the same family as ArcFace and CosFace. All embeddings lie on a hypersphere; during training, embeddings of the same person are pulled together and embeddings of different people are pushed apart by an angular margin. SFace controls these two forces with sigmoid curves, which makes training more robust to noisy labels. As a result, two photos of the same person produce embeddings pointing in almost the same direction, even with different lighting, expression or age.



Figure 4: From an aligned face to a 128-number embedding, compared with cosine similarity

4.6 Step 4 — Matching with cosine similarity

Each embedding is normalised to length 1 (L2 normalisation). The similarity of two embeddings a and b is the **cosine of the angle** between them:

$$\text{cosine_similarity}(a, b) = (a \cdot b) / (|a| \cdot |b|) \quad \text{range: } -1 \text{ (opposite)} \dots 1 \text{ (identical)}$$

All templates of all registered students are held in memory as one matrix, the gallery. One matrix multiplication gives the similarity of the new face to every template at once. The best score per student is taken, and a match is accepted only if **both** conditions hold:

- **Threshold:** the best similarity is at least **0.45**. Otherwise the person is **Unknown**.
- **Margin:** the best student beats the second-best student by at least **0.05**. Otherwise the decision is ambiguous and nothing is marked. This protects against look-alike students.

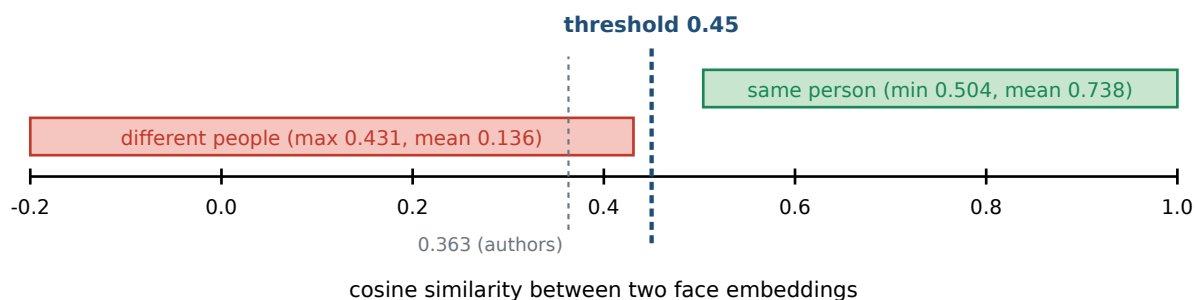


Figure 5: Measured similarity ranges on real photos and the chosen threshold

The model's authors suggest 0.363 for comparing two photos (1:1 verification). Attendance is **1:N identification**: a stranger is compared with every student, so the chance that one of them scores above a low threshold grows with class size. On real data, 0.363 accepted 25% of strangers; 0.45 accepted none while still identifying 100% of registered students (Chapter 13). The threshold can be changed in Admin → System settings and re-tuned with the evaluation script on your own students.

4.7 Step 5 — Tracking faces between frames

A student stays in view for several frames. A **tracker** links each new detection to the face seen in the previous frame by comparing boxes with **IoU** (Intersection over Union: overlap area divided by the union area). If $\text{IoU} \geq 0.3$ it is the same track; tracks unseen for 3 seconds are removed.

Real-data testing revealed another problem: if one student leaves and another sits in the same seat within 3 seconds, the boxes overlap, and the new student would inherit the old track and never be marked. The tracker therefore also compares embeddings: if the new face's similarity to the track's running average drops below 0.3, it is a different person and a new track is started.

4.8 Step 6 — Multi-frame voting

Single frames can be blurry or badly lit. Each track keeps the identity decisions of its last 5 frames, and a student is **confirmed only when 3 of the last 5 frames agree**. This majority vote removes one-frame errors and is an important reason why the system made no wrong-name marks in testing.

4.9 Enrolment: how students are registered

During enrolment 3–5 images of the student are captured with the webcam (or uploaded). For each image the system detects faces and takes the **dominant face**: small background faces are ignored, but two faces of similar size are rejected. It then computes the embedding and stores it **encrypted**. Two AI checks protect the gallery:

- **Same-person check:** the images of one enrolment must be similar to each other; otherwise the enrolment is rejected (wrong photos mixed up).
- **Duplicate-identity check:** if the new face already matches another registered student, it is rejected (“already registered as ...”). This stops the same person from being enrolled twice.

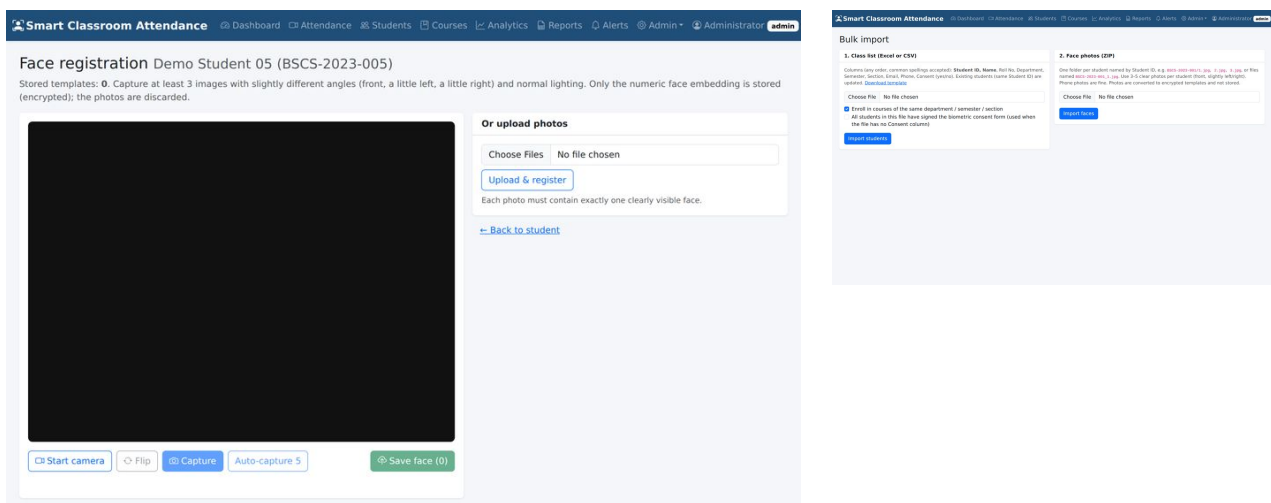


Figure 6: Face registration page (left) and bulk import of class lists and face photos (right)

5. Liveness Detection and Anti-Spoofing

Face recognition alone can be fooled: holding up a printed photo or a phone showing a student's picture would mark that student present. This is called a **presentation attack** or spoofing. The system uses two complementary liveness methods and lets the administrator choose how strict to be.

5.1 Method 1 — Anti-spoofing CNN (passive)

The main method is a deep-learning classifier based on the **MiniFASNet** architecture from the Silent-Face-Anti-Spoofing project, trained on the **CelebA-Spoof** dataset with three classes: live, print and replay (screen). It looks at fine texture and colour cues that humans hardly notice: paper grain, printing dots, screen pixels and moiré patterns, reflections and missing depth shading.

- Input: a square crop of 1.5× the face box, resized to 128×128 pixels (exactly as during training).
- Output: probability P(live). Over at least 3 frames the median is taken: $\geq 0.7 \rightarrow$ **live**, $< 0.3 \rightarrow$ **spoof**, in between $\rightarrow$ keep checking.
- It is **passive**: students only look at the camera; no movement is required. It takes about 5 ms per face.

Input	P(live)	Decision
64 genuine photos of real people	median 1.00, minimum 0.935	all 64 live ✓
Real person, webcam (Silent-Face sample)	0.999	live ✓
Printed photo attack (sample)	0.015	spoof ✓
Screen replay attack (sample)	0.001	spoof ✓

Table 8: Anti-spoofing CNN on real inputs

5.2 Method 2 — Head-movement test (own algorithm)

The second method is a geometric algorithm designed for this project. A printed photo or a screen is **flat**. Take three landmarks (eye 1, eye 2 and the mouth centre) as a coordinate frame and express the nose tip in that frame:

$$\text{nose} = \text{eye1} + a \cdot (\text{eye2} - \text{eye1}) + b \cdot (\text{mouth} - \text{eye1})$$

When a flat picture is moved, rotated or tilted in front of the camera, its image changes (approximately) by an **affine transformation**, and affine transformations preserve such coordinates. So for a photo, **(a, b) stays constant**. A real nose sticks out of the eye-mouth plane, so when a real head turns, (a, b) changes: about 0.12 for a 20° head turn. The student is asked to turn the head left, hold, then right; if (a, b) varies enough the face is live, and if it stays flat for 12 seconds it is flagged as a spoof.

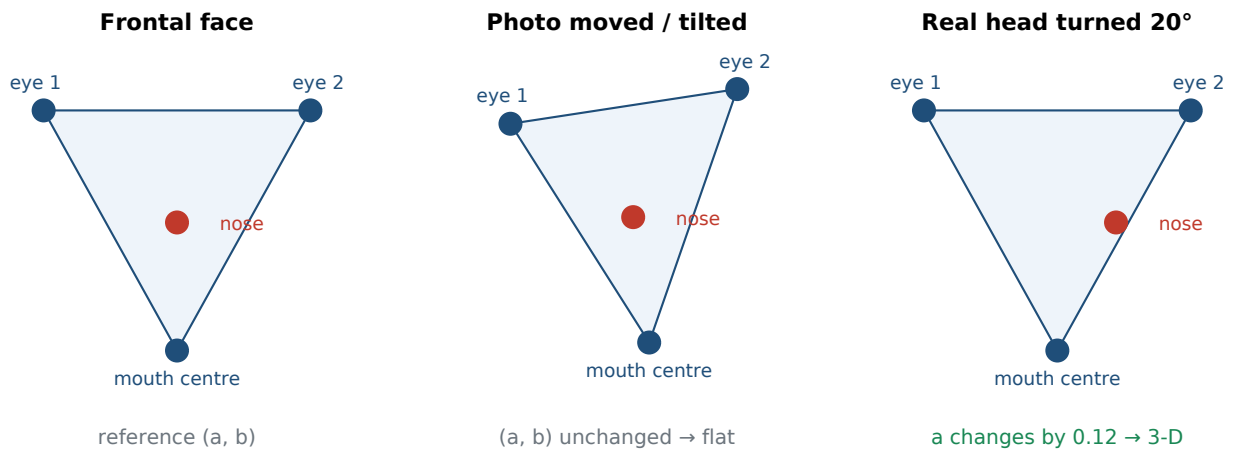


Figure 7: Geometric liveness: a flat photo keeps its nose coordinates, a real head does not

Measurements showed that the detector's landmarks are slightly biased when a face is rotated in-plane, so a moved photo showed some false “motion”. Four robustness measures were added: re-detecting the landmarks on an eye-aligned canonical crop, a 3-frame running median, a robust spread (10th–90th percentile), and ignoring frames rotated more than 15° in-plane. A blur check (variance of the Laplacian) rejects heavily blurred replays.

5.3 Liveness modes

Mode	How it works	When to use
auto / cnn (default)	Anti-spoofing CNN only; passive	Normal classes: fastest, no instructions for students
motion	Head-movement test only	If the CNN model is not installed
cnn+motion	Both must pass	When proxy attendance is a serious problem (strictest)

Table 9: Liveness modes (Admin → System settings)

Honest limitation

No liveness method is perfect. The head-movement test alone can be fooled by aggressive photo waving in about 25% of simulated attempts, and by a video of the student turning their head; that is why the CNN is the default. The CNN has been validated here on 64 genuine photos and 2 real attack samples. A 3-D silicone mask is outside the scope of both methods. This belongs in the limitations section of the report.

6. Decision Logic: From Recognition to Attendance

6.1 States of a face on the live screen

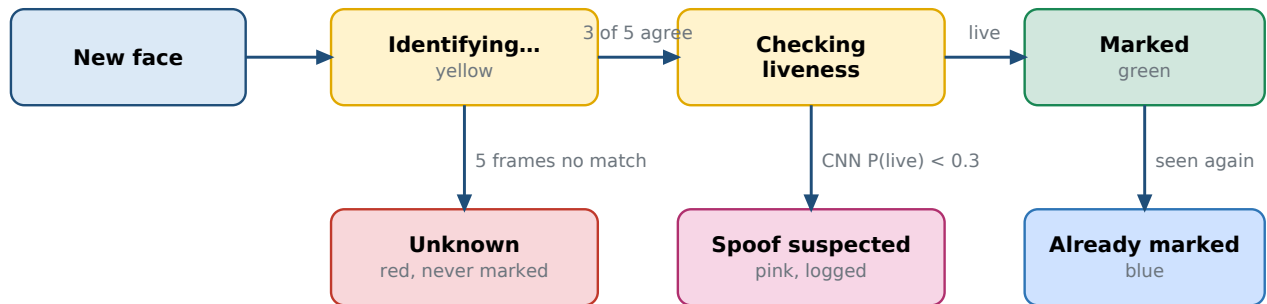


Figure 8: How a face moves between states on the live attendance screen

6.2 Attendance status rules

When a confirmed, live face belongs to a student enrolled in the session's course, the arrival time decides the status. The windows are set per session (defaults from the system settings):

Arrival (minutes after start)	Status	Example (class at 09:00)
0 – 10 (present window)	Present	09:03 → Present
10 – 20 (late window)	Late	09:14 → Late
after 20	Absent (with note “arrived after late cut-off”)	09:25 → Absent
never seen, session closed	Absent (method: auto-absent)	—
set by teacher	Excused / Leave	medical leave

Table 10: Status rules

6.3 Duplicate prevention

A student recognised five times in one class must get only one record. This is enforced twice:

- **In the service layer:** before inserting, the system checks whether a record already exists and answers “attendance already marked”.
- **In the database:** the attendance table has a **UNIQUE(student_id, session_id)** constraint. Even if two camera frames arrive at the same moment, the database refuses the second insert.

6.4 Other rules

- A recognised student who is **not enrolled** in the course is not marked (“not enrolled in this course”).
- A **closed** session accepts no more marks; closing marks all missing enrolled students Absent.
- Teachers can **override** any status (Excused, Leave, corrections) with a note; the method is recorded as manual.
- **Attendance percentage** = $(\text{Present} + \text{Late}) \div (\text{all records} - \text{Excused} - \text{Leave}) \times 100$. Excused and leave do not count against the student.
- **Alerts:** after each closed session, students below the threshold (default 75%, at least 3 classes) get one live warning per course. The warning is updated after every class, removed automatically once attendance recovers, and optionally e-mailed.

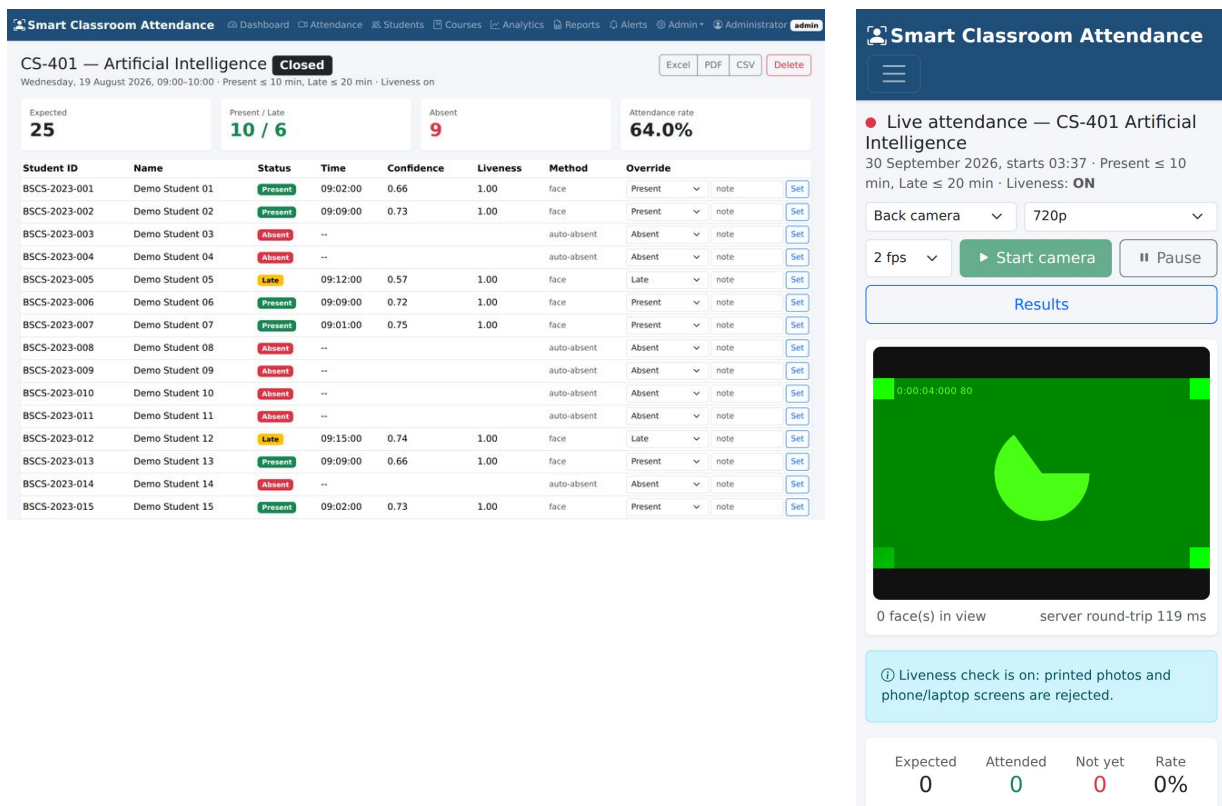


Figure 9: Session results with manual override (left) and live attendance on a phone with the back camera (right)

7. Database Design

The data is stored in a normalised relational database with ten tables. SQLAlchemy maps each table to a Python class, so the same code runs on SQLite (default, a single file, no installation) and PostgreSQL (for a server deployment).

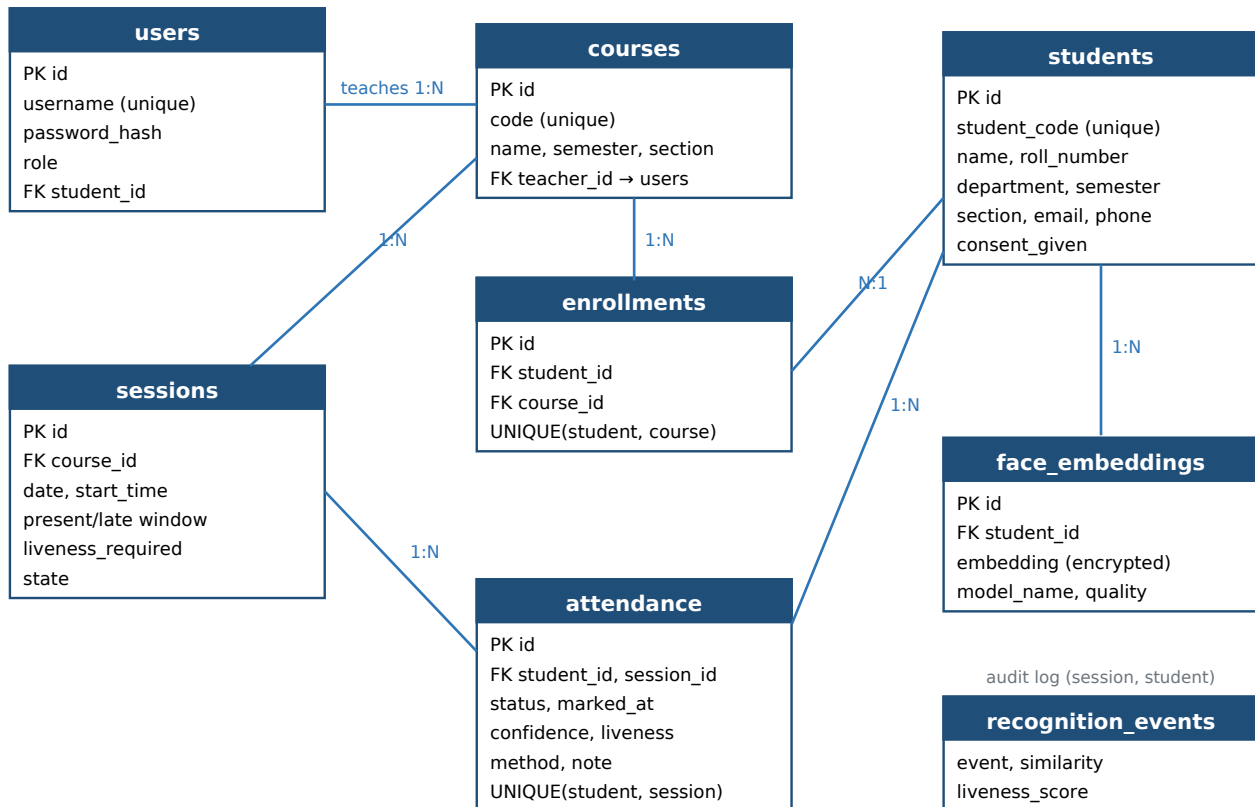


Figure 10: Entity-relationship diagram (main tables)

Table	Purpose	Important columns / constraints
users	Login accounts	username UNIQUE, password_hash (PBKDF2), role, optional link to a student
students	Student profiles	student_code UNIQUE, name, roll no., department, semester, section, consent_given
face_embeddings	Face templates	student_id FK (cascade delete), embedding encrypted, model_name, quality
courses	Courses	code UNIQUE, semester, section, teacher_id FK
enrollments	Student ↔ course	UNIQUE(student_id, course_id)
sessions	One class / lecture	course, date, start time, duration, present / late window, liveness_required, state
attendance	Attendance records	UNIQUE(student_id, session_id), status, marked_at, confidence, liveness_score, method, note
recognition_events	AI audit log	event (marked / duplicate / unknown / spoof), similarity, liveness score
notifications	Low-attendance alerts	student, course, message, is_read
app_settings	Configurable rules	key → value (thresholds, windows, liveness mode)

Table 11: Database tables

Design decisions: foreign keys are enforced; deleting a student deletes all their biometric data (cascade); the model name is stored with each template, so templates from a different model are ignored automatically; raw photos are never stored.

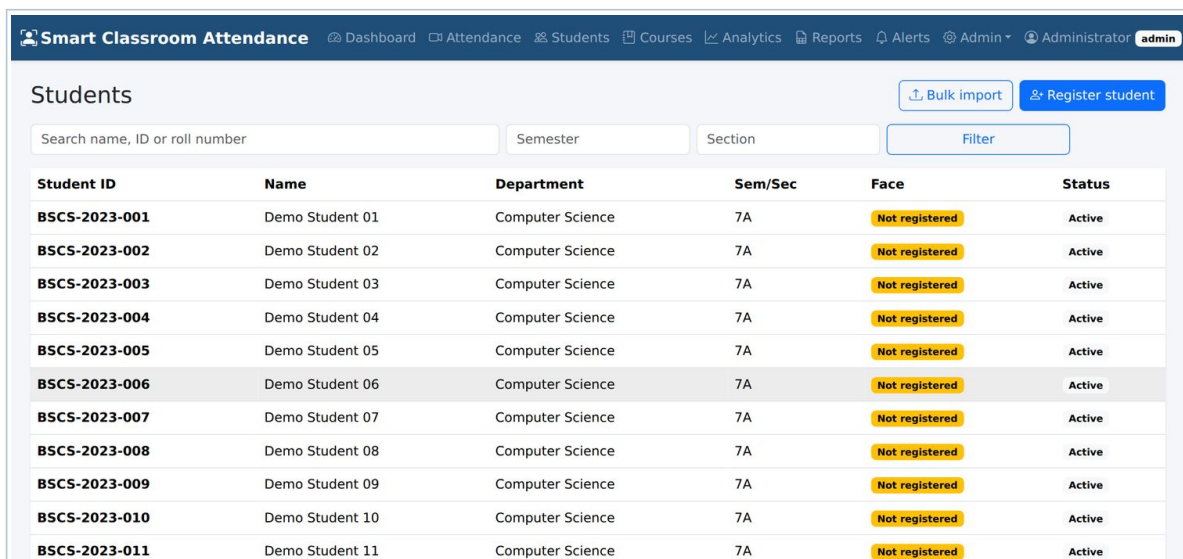
8. Module-by-Module Walkthrough

8.1 Login and roles

Users log in with a username and password. Passwords are stored as PBKDF2-SHA256 hashes (240 000 iterations, random salt). After five failed attempts within 15 minutes, the account is locked for that address. A signed session cookie keeps the user logged in for 8 hours. Every page and API endpoint checks the role before doing anything.

8.2 Student management and bulk import

Students can be registered one by one or imported from the registrar's **Excel or CSV** class list. The importer recognises common column names (Student ID, Reg No, Name, Roll No, Semester, Section, Email, Phone, Consent), updates existing students instead of duplicating them, reports unknown columns and bad rows, and enrolls students automatically in matching courses. Face photos can be imported as a ZIP with one folder per Student ID.



Student ID	Name	Department	Sem/Sec	Face	Status
BSCS-2023-001	Demo Student 01	Computer Science	7A	Not registered	Active
BSCS-2023-002	Demo Student 02	Computer Science	7A	Not registered	Active
BSCS-2023-003	Demo Student 03	Computer Science	7A	Not registered	Active
BSCS-2023-004	Demo Student 04	Computer Science	7A	Not registered	Active
BSCS-2023-005	Demo Student 05	Computer Science	7A	Not registered	Active
BSCS-2023-006	Demo Student 06	Computer Science	7A	Not registered	Active
BSCS-2023-007	Demo Student 07	Computer Science	7A	Not registered	Active
BSCS-2023-008	Demo Student 08	Computer Science	7A	Not registered	Active
BSCS-2023-009	Demo Student 09	Computer Science	7A	Not registered	Active
BSCS-2023-010	Demo Student 10	Computer Science	7A	Not registered	Active
BSCS-2023-011	Demo Student 11	Computer Science	7A	Not registered	Active

Figure 11: Student list with face-registration status

8.3 Courses and enrolment

Each course has a code, name, semester, section and teacher. Students can be enrolled individually or all at once (“enrol all of semester 7A”). The course page shows every student's attendance in that course.

8.4 Sessions and live attendance

A teacher creates a session (course, date, start time, duration, present and late windows, liveness on or off) and starts the live camera. The browser captures a frame every 500 ms (adjustable to 1-3 frames per second, 480p to 1080p, front or back camera) and sends it to `/api/sessions/{id}/frame`. The answer contains, for every face, the box, the name, the state and a message, which the page draws over the video.

Counters show expected, attended and not-yet-seen students and the attendance rate.

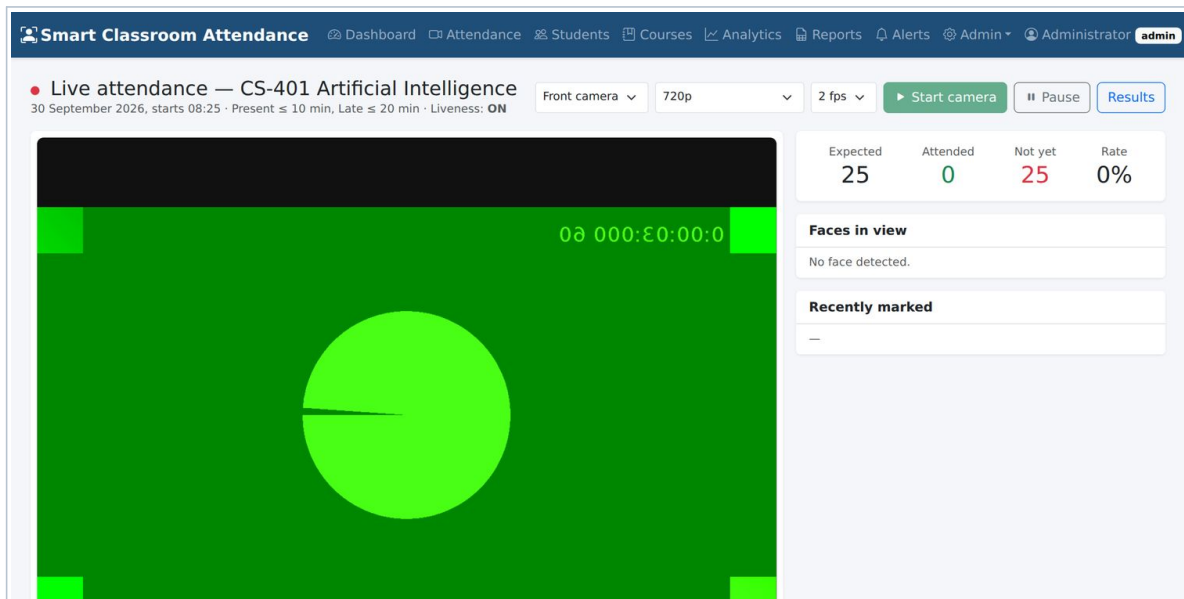


Figure 12: Live attendance page on a PC (camera controls, overlay, counters and recently marked list)

8.5 Analytics

The analytics page shows daily attendance for the last 30 days, weekly and monthly trends, course-wise attendance, the highest and lowest attendance students, and absences by weekday (useful for spotting problem days).

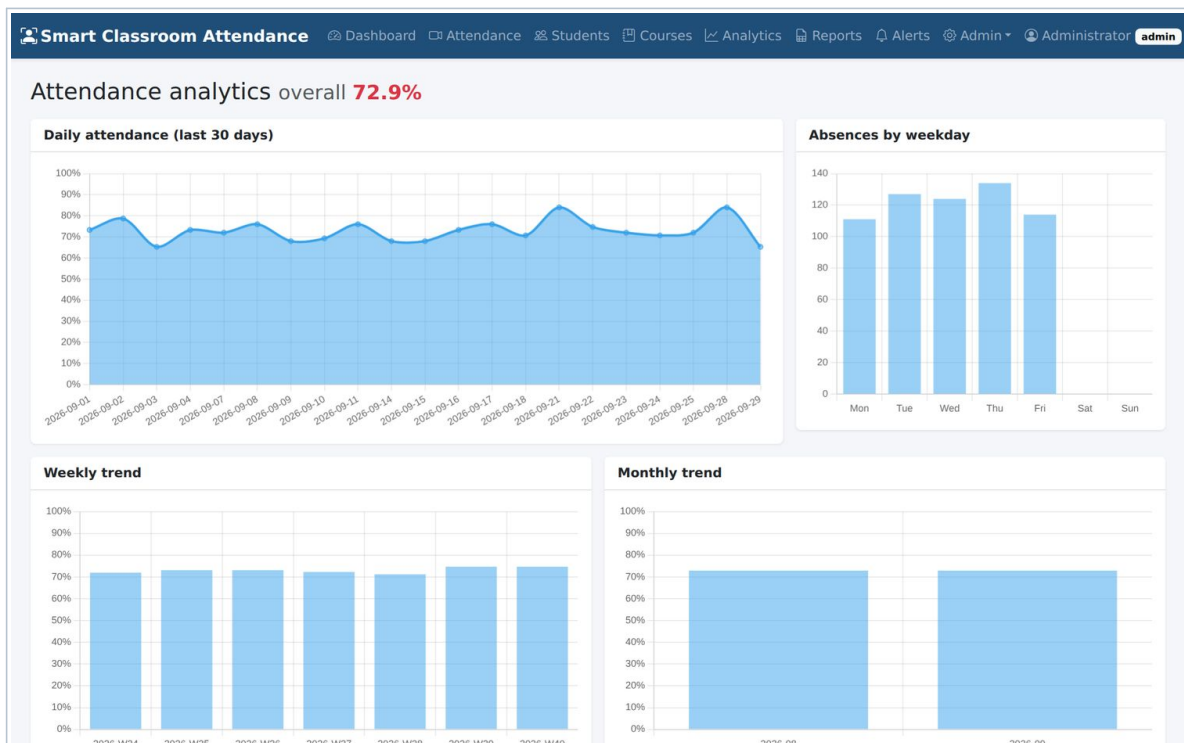
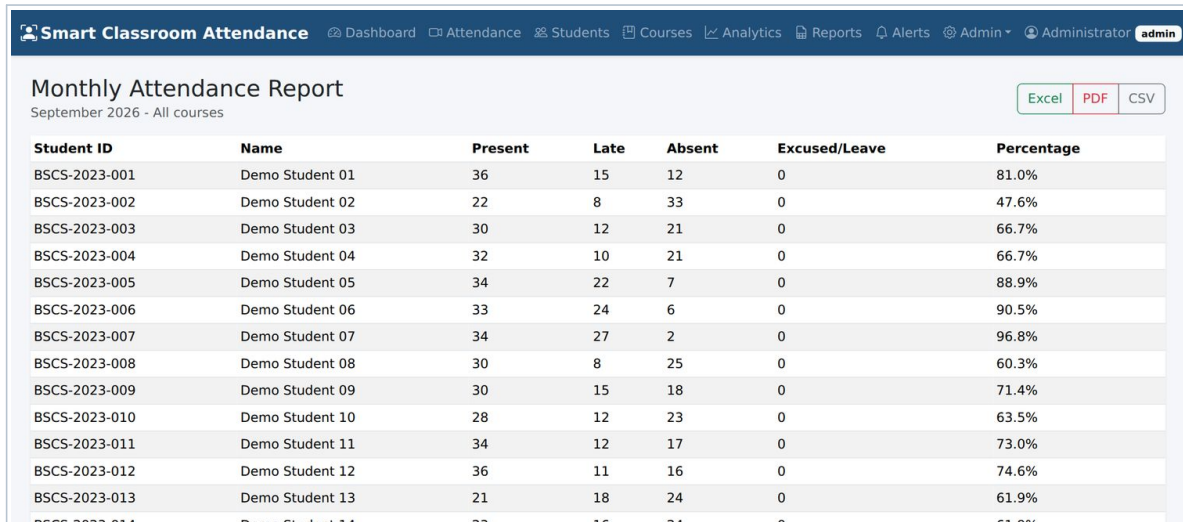


Figure 13: Analytics page

8.6 Reports

Three kinds of report can be viewed on screen or exported to Excel (.xlsx), PDF and CSV: a **daily** report (every student of every session on one date), a **monthly** report (present, late, absent, excused and percentage per student) and a **session** report.

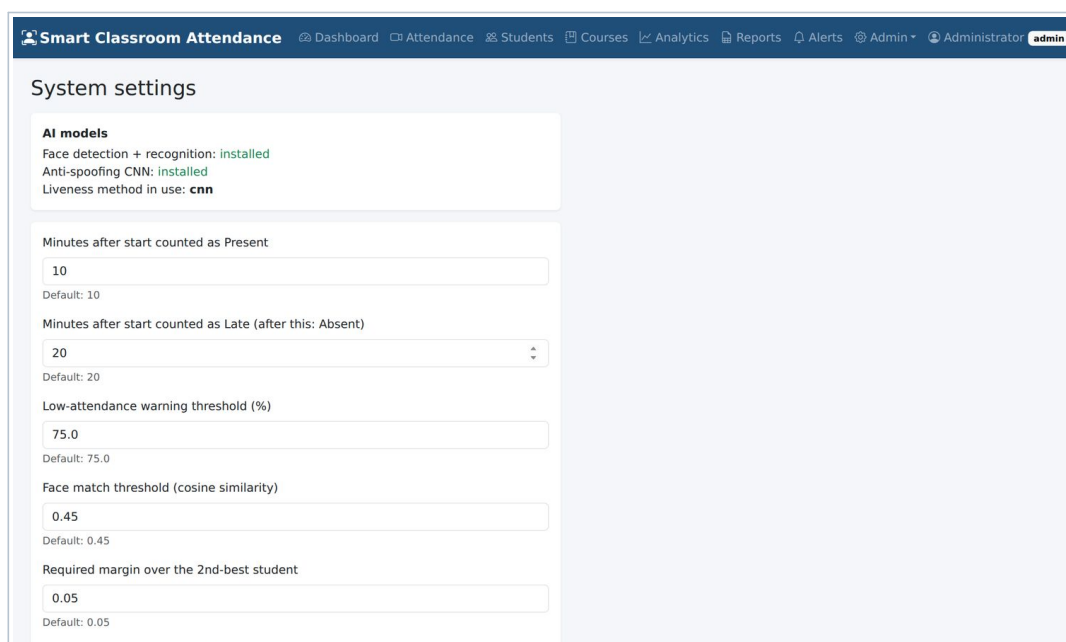


Student ID	Name	Present	Late	Absent	Excused/Leave	Percentage
BSCS-2023-001	Demo Student 01	36	15	12	0	81.0%
BSCS-2023-002	Demo Student 02	22	8	33	0	47.6%
BSCS-2023-003	Demo Student 03	30	12	21	0	66.7%
BSCS-2023-004	Demo Student 04	32	10	21	0	66.7%
BSCS-2023-005	Demo Student 05	34	22	7	0	88.9%
BSCS-2023-006	Demo Student 06	33	24	6	0	90.5%
BSCS-2023-007	Demo Student 07	34	27	2	0	96.8%
BSCS-2023-008	Demo Student 08	30	8	25	0	60.3%
BSCS-2023-009	Demo Student 09	30	15	18	0	71.4%
BSCS-2023-010	Demo Student 10	28	12	23	0	63.5%
BSCS-2023-011	Demo Student 11	34	12	17	0	73.0%
BSCS-2023-012	Demo Student 12	36	11	16	0	74.6%
BSCS-2023-013	Demo Student 13	21	18	24	0	61.9%

Figure 14: Monthly report with export buttons

8.7 Administration

- **Users:** create teacher, admin and student accounts, disable accounts, reset passwords.
- **System settings:** attendance windows, low-attendance threshold, match threshold and margin, votes required, liveness mode and thresholds. The page also shows which AI models are installed.
- **AI audit log:** every automatic decision (marked, duplicate, unknown, spoof) with similarity and liveness score, useful for investigating disputes.



System settings

AI models

Face detection + recognition: installed

Anti-spoofing CNN: installed

Liveness method in use: cnn

Minutes after start counted as Present

Default: 10

Minutes after start counted as Late (after this: Absent)

Default: 20

Low-attendance warning threshold (%)

Default: 75.0

Face match threshold (cosine similarity)

Default: 0.45

Required margin over the 2nd-best student

Default: 0.05

Figure 15: System settings: AI model status and configurable thresholds

9. Web Application and Mobile App

9.1 One code base, two kinds of app

The user interface is a responsive web application: the same pages adapt to a large monitor or a phone screen. On phones it is additionally a **Progressive Web App (PWA)**. A PWA is a website that the browser can install like a native app: it gets its own icon on the home screen, opens full-screen without the browser bar, and can work partially offline. Three parts make this possible:

- **Web app manifest** (`manifest.webmanifest`): app name, icons (192 and 512 px, maskable), colours, start page and shortcuts (“Take attendance”, “My attendance”).
- **Service worker** (`sw.js`): a background script that caches the app shell (styles, scripts, icons) and shows an offline screen when the server cannot be reached. Attendance data and camera frames are never cached.
- **HTTPS**: browsers only allow camera access and app installation on secure connections.

Smart Classroom Attendance

AI Attendance Dashboard

[Start attendance](#)

Students

25

0 with registered face

Present today

0

0 of them late

Absent today

0

of 25 expected

Today's attendance

0.0%

Overall: **72.9%**

Attendance trend (last 14 days)

Today's sessions

Course	Time	State	
CS-401 - Artificial Intelligence	08:25	Active	Live Details

Smart Classroom Attendance

Demo Student 05 BSCS-2023-005

Computer Science · Semester 7 · Section A

Total classes

90

Present

48

+ 32 late

Absent

10

Attendance

88.9%

By course

Course	Attended	%
CS-401 Artificial Intelligence	26/30	86.7%
CS-403 Computer Vision	26/30	86.7%
CS-405 Software Engineering	28/30	93.3%

History

Date	Course	Status	Time
29-09-2026	CS-401	Present	09:01
29-09-2026	CS-403	Present	11:02
29-09-2026	CS-405	Present	14:03
28-09-2026	CS-401	Present	09:02
28-09-2026	CS-403	Present	11:09
28-09-2026	CS-405	Late	14:18
25-09-2026	CS-401	Late	09:12

Smart Classroom Attendance

Mobile app setup

Use the system on Android phones, iPhones and tablets: teachers take attendance with the phone camera, students check their own attendance. The app installs from the browser, with no app store needed.

Scan with the phone camera, or type:
https://192.0.2.2:8443

The phone must be on the **same Wi-Fi** as this server.

Android (Chrome)

- Trust the server (once per phone):** [download the certificate](#), then open *Settings* → *Security* → *More security settings* → *Encryption & credentials* → *Install a certificate* → *CA certificate* and choose *attendance-ca.crt*. On some phones the path is *Settings* → *Security* → *Install from storage*.
- Open the address above in **Chrome** and log in.
- Tap **Install app** (top bar) or Chrome menu : → **Install app** / **Add to Home screen**.
- Open **Attendance** from the home screen. It runs full-screen like a normal app.

iPhone / iPad (Safari)

- Trust the server (once per device):** open this page in **Safari**, [download the certificate](#) and tap *Allow*. Then go to *Settings* → *Profile Downloaded* → *Install*, and after that *Settings* → *General* → *About* → *Certificate Trust Settings* and switch it **on**.
- Open the address above in **Safari** and log in.
- Tap **Share** → **Add to Home Screen**.
- Open **Attendance** from the home screen.

Using the app

- Teacher:** Attendance → Create session → *Create & start* → choose *Back camera* → point the phone at the class. On a stand or tripod, 2 frames per second, 720p.
- Face registration on a phone:** Students → student → Register face → *Flip* to choose the camera.
- Student:** log in with the Student ID to see attendance percentage, history and warnings.
- If you skip the certificate step, the browser shows a security warning. Tap *Advanced* → *Proceed*; the camera still works, but the app cannot be installed and each visit shows the warning again.

Figure 16: Mobile app: teacher dashboard, student's own attendance, and the mobile setup page with QR code

9.2 Making phones trust the server

A server in a classroom has no public domain name, so it cannot get a normal HTTPS certificate. The launcher (`start-mobile.bat / run.py --lan`) therefore creates a small **local certificate authority** (CA) and a server certificate for the PC's network address, signed by that CA. Each phone installs the CA certificate once from the Mobile setup page. After that the phone trusts the server, the camera works and the app can be installed. If the PC's IP address changes, only the server certificate is re-issued, so phones keep working.

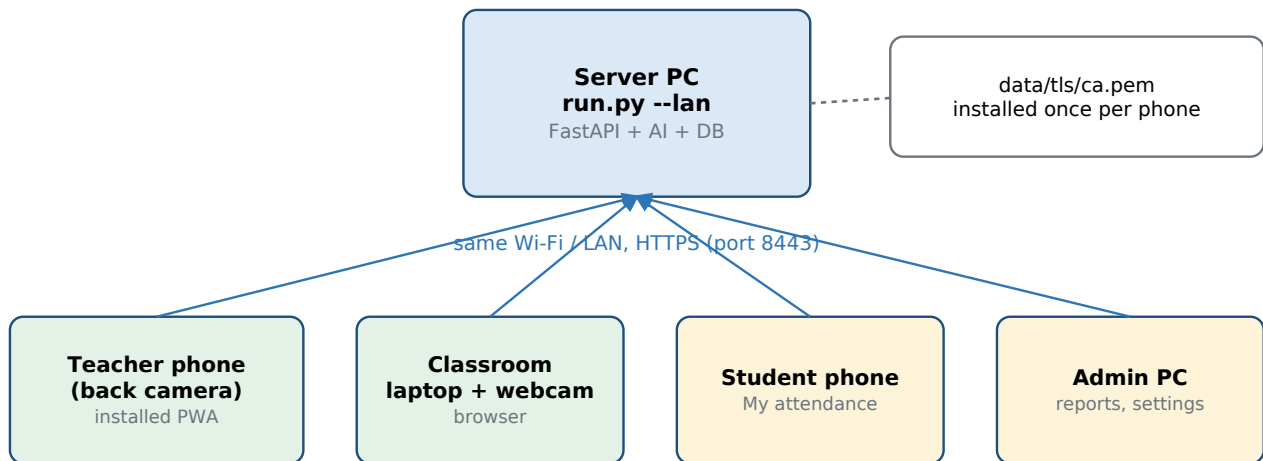


Figure 17: Deployment in a department: one server, many devices on the same network

9.3 Verified behaviour

- In a phone-sized Chromium browser, Chrome reported the app as **installable with no errors**; the service worker was active.
- The back camera streamed frames to the server correctly (not mirrored); the front camera is mirrored like a selfie.
- With the server switched off, the installed app showed the “Server not reachable” screen instead of a browser error.
- A connection trusting only the downloaded CA certificate was accepted for both the network address and localhost.

10. Security, Privacy and Ethics

Facial templates are **biometric data**, one of the most sensitive categories of personal data. The system is designed with privacy by design:

Data	Stored?	Protection
Enrolment photos	No	decoded in memory, converted to an embedding, discarded
Live camera frames	No	processed in memory only
Face embeddings	Yes	encrypted with Fernet (AES-128-CBC + HMAC-SHA256); key derived from SECRET_KEY
Passwords	Hash only	PBKDF2-HMAC-SHA256, 240 000 iterations, random salt
Attendance and audit log	Yes	role-based access; no images in the audit log

Table 12: What is stored and how it is protected

- **Consent gate:** faces cannot be enrolled unless consent is recorded; a consent form template is in the deployment guide.
- **Right to erasure:** “Delete face data” removes all templates; deleting a student removes all their data.
- **Access control:** teachers see only their own courses and students only their own records.
- **Human in the loop:** teachers can override any automatic decision, and every AI decision is logged.
- **Other measures:** login lock-out, HTTPS for phones, SameSite session cookies, safe redirects, no export of templates.
- **Ethics:** the system only runs during a session started by a teacher (no continuous surveillance); pre-trained face models can have demographic bias, so accuracy should be measured per group on your own data.

11. Installation and Running the System

11.1 Requirements

- Windows 10/11, Linux or macOS; 4-core CPU and 8 GB RAM recommended; no GPU needed.
- Python 3.10 or newer (3.10–3.14 tested). On Windows tick “**Add python.exe to PATH**” during installation.
- Internet for the first start (libraries and the three AI models, about 40 MB).
- A webcam, or a phone on the same Wi-Fi network.

11.2 First start (Windows)

1. Download or clone the project from GitHub (branch main).
2. Double-click **start.bat**. The first start creates a virtual environment, installs the libraries and downloads the models (3–10 minutes).
3. When the window shows Open: `http://127.0.0.1:8000`, open that address in Chrome or Edge.
4. Log in with **admin / admin123** and change the password immediately.
5. Optional demo data: `.venv\Scripts\python scripts\seed_demo.py (teacher / teacher123)`.

Linux / macOS: `./start.sh`. For phones: **start-mobile.bat** (or `./start.sh --lan`), then open `https://localhost:8443/mobile` on the PC and follow the QR-code instructions.

11.3 Useful commands

<code>python scripts/download_models.py</code>	download / verify the AI models
<code>python scripts/import_data.py --students list.xlsx</code>	import a class list
<code>python scripts/import_data.py --faces photos/</code>	import face photos (folder or ZIP)
<code>python scripts/evaluate.py --data dataset --out results</code>	evaluate recognition (FAR, FRR, EER ...)
<code>python scripts/evaluate_liveness.py --data liveness</code>	evaluate anti-spoofing (APCER, BPCER)
<code>python scripts/webcam_attendance.py --session 3</code>	desktop mode with a USB / IP camera
<code>python scripts/backup.py</code>	back up the database
<code>pytest -q</code>	run the 46 automated tests
<code>docker compose up -d</code>	server deployment with PostgreSQL

11.4 Troubleshooting

Problem	Solution
python is not recognized	Reinstall Python with “Add python.exe to PATH”; disable the Microsoft Store app alias
venv error with Python 3.14	Handled automatically: start.bat falls back to virtualenv or a per-user installation

Problem	Solution
Camera does not start	Use Chrome / Edge / Safari and allow the camera; phones need the https address
Missing model files	Check the internet connection, run scripts/download_models.py
A student is not recognised	Register 3-5 more photos in good light; check the threshold in settings
Phones cannot connect	Same Wi-Fi; allow Python in Windows Firewall; use start-mobile.bat

Table 13: Common problems

12. Testing

Testing was done at three levels: automated unit and integration tests, browser tests, and evaluation with real face data (Chapter 13).

Test group	What is verified
Liveness geometry	A simulated 3-D head versus a flat photo; affine invariance of (a, b); jitter filter; roll gate; CNN, motion and combined modes
Matching and tracking	Unknown and ambiguous faces; IoU tracking; a new person in the same seat gets a new track; 3-of-5 voting
Attendance rules	Present / Late / Absent windows; duplicate prevention including the database constraint; closing sessions; not-enrolled students; overrides
Analytics and alerts	Percentage formula; rankings; trends; alerts created, updated and resolved when attendance recovers
Import and enrolment	Excel and CSV class lists; ZIP photos; consent; dominant face; duplicate identity
Security	Password hashing; encrypted templates; login lock-out; role-based access
Reports and mobile	Excel / PDF / CSV exports; PWA manifest, service worker, icons; local CA and IP change
End-to-end web flow	Register → enrol → live frames → marked → duplicate → unknown → spoof → close → reports
Real-data end-to-end	Whole system through the web API on real photos (optional, Chapter 13)

Table 14: Automated tests (46 tests, all passing)

The automated tests use a deterministic fake vision backend, so they run in about 12 seconds without models or a camera. The real models are exercised by the real-data test and the evaluation scripts.

13. Evaluation and Results

Data used

61 real photographs of 13 people (varied lighting, pose, age, glasses, resolution from 0.3 to 12 megapixels) with 520 labelled pairs (140 same person, 380 different people), from the open-source deepface test set; real attack samples from the Silent-Face-Anti-Spoofing project; measured on a normal CPU without GPU. This is a small public benchmark: repeat the measurements on your own students for the final report (§1 of docs/EVALUATION.md describes the protocol).

13.1 Metrics used

Metric	Meaning
Detection rate	Share of images in which the face was found
Identification rate	Registered person accepted and correctly named
FRR – False Rejection Rate	Registered person not recognised (missed)
FAR – False Acceptance Rate	Attendance would go to the wrong person (stranger accepted or misidentified)
ROC AUC	Probability that a same-person pair scores higher than a different-person pair (1.0 = perfect)
TAR @ FAR	Share of same-person pairs accepted at a threshold where only the given share of impostors pass
APCER / BPCER	Attacks accepted as live / real people rejected as spoofs (ISO/IEC 30107-3)

Table 15: Evaluation metrics

13.2 Face detection

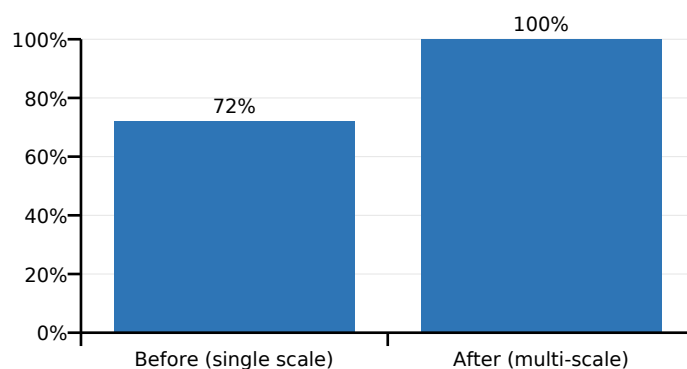


Figure 18: Face detection rate on 61 real photos, before and after multi-scale detection

13.3 Face recognition

Test	Result
1:1 verification, 520 pairs, accuracy at 0.363	99.81%
ROC AUC	1.000
TAR at FAR = 0.1%	100%
Mean similarity: same person / different people	0.738 / 0.136
Minimum same-person / maximum different-person similarity	0.504 / 0.431
1:N identification at 0.45 (10 enrolled, 3 strangers)	100% identified, 0% strangers accepted, 0% misidentified
Rank-1 accuracy (closed set)	100%

Table 16: Recognition results on real photos

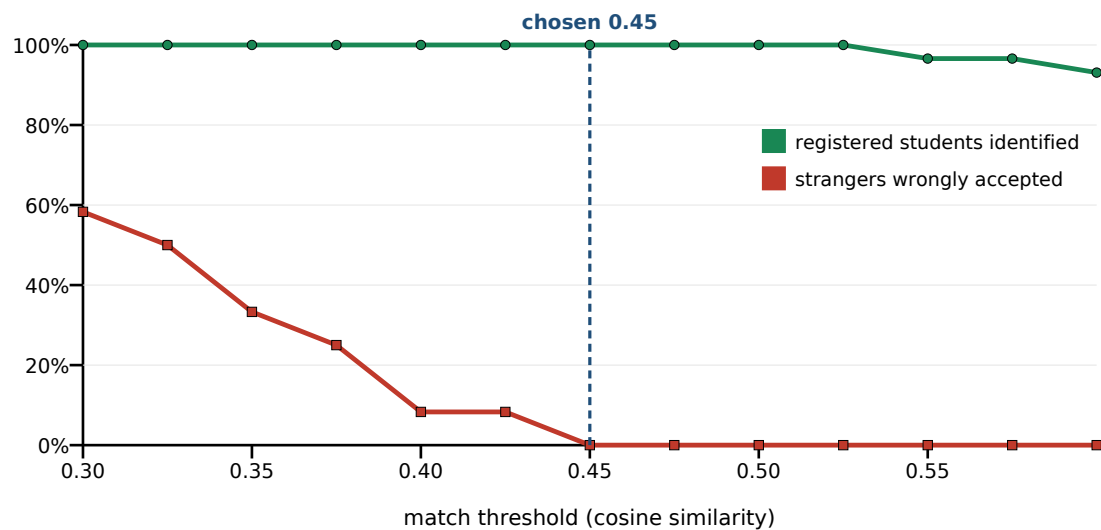


Figure 19: Threshold sweep: identification rate and stranger acceptance (1:N, real photos)

13.4 Liveness

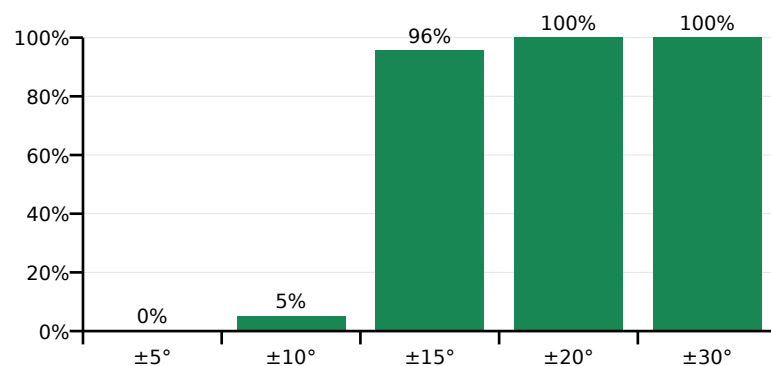


Figure 20: Head-movement test: genuine users accepted vs. head-turn amplitude (model-based simulation, 200 trials each)

Photo attack against head-movement test only	Accepted (APCER)
Gentle waving ($\leq 10^\circ$ rotation, small tilt)	0 / 40 = 0%
Moderate ($\leq 20^\circ$, medium tilt)	0 / 40 = 0% (10% before robustness fixes)
Aggressive ($\leq 35^\circ$, strong tilt)	10 / 40 = 25% (40% before fixes)
Real print and replay samples against the anti-spoofing CNN (default mode)	2 / 2 rejected

Table 17: Liveness results

13.5 End-to-end system test on real data

The complete system was driven through its web API exactly as the browser does: a CSV class list of 10 people and a ZIP with 2 photos each were imported, a session was started with liveness on, and every remaining photo was streamed as six noisy 1280×720 webcam-like frames. Strangers and the two attack samples were also shown.

Outcome	Result
Registered student marked with the correct name	29 / 30
Registered student marked with a wrong name	0
Still “checking liveness” after 6 frames (not rejected)	1
Stranger photos marked present	0 / 11
Real print / replay attacks rejected	2 / 2
Session closed, absentees auto-marked, Excel report exported	✓

Table 18: End-to-end results

13.6 Speed

Frame size (7 faces in view, CPU)	Detection	Per face (embedding + anti-spoofing)	Total per frame
1280 × 720 (720p)	72 ms	≈ 27 ms	258 ms
1920 × 1080 (1080p)	152 ms	≈ 30 ms	363 ms

Table 19: Processing time; the live page allows 500 ms per frame (2 frames per second)

14. Problems Found and Solved

A valuable part of the project was testing on real data and fixing what failed. These are good examples to mention in the presentation because they show engineering judgement rather than just integrating libraries.

Problem discovered	Cause	Solution	Result
28% of real photos: no face found	Detector trained on small/medium faces	Multi-scale detection (640 px + full resolution)	100% detected
25% of strangers accepted	Threshold 0.363 is for 1:1, not 1:N	Threshold 0.45 from the measured sweep, plus margin and voting	0% accepted, 100% identified
New student in the same seat never marked	IoU tracker kept the old identity	Split the track when the embedding changes	Test added
Moved photo looked “3-D”	Landmark bias under rotation	Canonical re-detection, median filter, robust spread, roll gate	Attack acceptance 40% → 25%; CNN added
Video replay not stoppable by geometry	Real 3-D motion in the video	Anti-spoofing CNN as the default	Replay sample rejected
Old alert showed an outdated percentage	Alert text frozen at creation	One live alert per course, updated or resolved each session	Test added
Windows setup failed on Python 3.14	Installer missing a venv file; one library had no 3.14 package	Fallback setup methods; plain uvicorn	Installs on Python 3.10–3.14

Table 20: Problems found during development and how they were solved

15. Limitations and Future Work

15.1 Limitations

- Accuracy was measured on a small public benchmark (13 people); a study with your own students is needed for final numbers.
- Anti-spoofing is validated on 64 genuine photos and 2 real attack samples; 3-D masks are not covered.
- Faces must be at least about 40 px wide; very large halls need 1080p cameras near the front or several cameras.
- On a CPU about 10–15 faces per frame are comfortable at 2 frames per second.
- The mobile app needs the phone on the same network as the server and a one-time certificate installation.
- The models are pre-trained; their accuracy may differ between demographic groups.

15.2 Future work

- Fine-tune the recognition threshold per class automatically from enrolment photos.
- Train or fine-tune the anti-spoofing model on locally recorded attacks; add depth or infrared cameras.
- GPU inference and multiple cameras per room; Redis for sharing trackers between servers.
- Publish the mobile app on Google Play (Trusted Web Activity) with a public HTTPS address.
- Integration with the university LMS/ERP; SMS alerts; QR or geofence check-in as a second factor.

16. Presentation and Viva Guide

16.1 Suggested slides (15-18 minutes)

Slide	Content
1. Title	Project title, names, supervisor. One sentence: "AI system that marks attendance by recognising faces and rejecting photos."
2. Problem	Roll call wastes time; proxy attendance; manual registers are error-prone and hard to analyse.
3. Objectives	Automatic recognition, unknown rejection, anti-spoofing, rules, reports, web + mobile, measured accuracy.
4. System overview	End-to-end flow figure (Chapter 1) and a one-minute walk through a lecture.
5. Architecture	Layered architecture figure and technology stack table (Chapter 3).
6. AI 1: detection	YuNet CNN, landmarks, multi-scale fix (72% → 100%).
7. AI 2: recognition	Embeddings, hypersphere (ArcFace-family) training, cosine similarity, threshold 0.45 (similarity-range figure, Chapter 4).
8. AI 3: liveness	Anti-spoofing CNN + own geometric method (geometry figure, Chapter 5); modes.
9. Decision logic	Tracking, 3-of-5 voting, Present/Late/Absent, duplicate prevention.
10. Database	ER diagram; unique constraints; encrypted templates.
11. Live demo	See §16.2 (3-4 minutes).
12. Mobile app	PWA installed on a phone; teacher and student views.
13. Results	Detection, verification 99.8%, 0% strangers, end-to-end 29/30, 0 wrong names, spoofs rejected, speed.
14. Problems solved	Problems-and-solutions table (Chapter 14): shows engineering on real data.
15. Security & ethics	Consent, no photos stored, encryption, roles, human override.
16. Limitations & future work	Honest limitations; next steps.
17. Conclusion	Objectives met; questions.

Table 21: Slide plan

16.2 Live demo script

- Before the viva:** start the server (start-mobile.bat), log in, create a course, register 2-3 group members' faces, and install the app on a phone. Keep a printed photo of one member ready.
- Show the **dashboard** and **analytics** with the demo data (30 seconds).
- Create & start** a session. Walk in front of the camera: the box turns yellow ("Identifying...") and then green with your name and "Present 10:03".
- Step out and in again: the box turns blue, "attendance already marked" (duplicate prevention).
- Hold up the **printed photo**: the box turns pink, "Spoof suspected". Show the entry in the AI audit log.

6. Ask an examiner (not registered) to look at the camera: red box **“Unknown”**.
7. Close the session: absentees appear automatically; export the Excel or PDF report.
8. Open the **installed app** on the phone: switch to the back camera and show a student's own attendance view.

Demo tips

Use good, even light on faces; place the camera at face height; keep a backup screen recording of the demo in case the room's network fails; know your numbers (99.8%, 0 strangers, 29/30, 0.45).

16.3 Likely viva questions and answers

Q: What is the difference between face detection and face recognition?

Detection finds where faces are (boxes and landmarks) without knowing who they are; recognition identifies the person by comparing the face's embedding with registered embeddings. We use YuNet for detection and SFace for recognition.

Q: What is a face embedding?

A vector of 128 numbers produced by a deep network, trained so that images of the same person give vectors pointing in nearly the same direction and different people give different directions. Comparison is done with cosine similarity.

Q: Why cosine similarity and not Euclidean distance?

The network is trained with a hypersphere (angular-margin) loss, so identity is encoded in the direction of the vector. For L2-normalised vectors, cosine similarity and Euclidean distance are equivalent rankings anyway, and cosine is cheap: one matrix multiplication for the whole class.

Q: Did you train the models?

No, we used pre-trained models, which is standard practice: training a recogniser needs millions of labelled faces and large GPUs. Our contribution is the integration, the fixes found on real data (multi-scale, threshold, track splitting), the liveness method and the evaluation.

Q: How did you choose the threshold 0.45?

From a measured threshold sweep on real photos: at the authors' 0.363, 25% of strangers were accepted in 1:N identification; at 0.45 none were, with 100% identification. The smallest same-person similarity was 0.504 and the largest different-person 0.431, so 0.45 lies in the gap.

Q: What happens if a stranger comes in front of the camera?

Their best similarity is below 0.45, so the face is labelled Unknown, logged in the audit trail and never marked. In testing, 0 of 11 stranger images were marked.

Q: How do you prevent a student from using a friend's photo?

Liveness detection: an anti-spoofing CNN classifies each face as live, print or screen replay; optionally a head-movement test checks for 3-D structure. Both real attack samples were rejected. Spoof attempts are logged.

Q: Explain your geometric liveness method.

We express the nose in the coordinate frame of the two eyes and the mouth centre. For a flat photo these coordinates are invariant under affine motion; a real head turning changes them because the nose is not in the eye-mouth plane. About 20 degrees of head turn changes them by 0.12, our threshold.

Q: How do you avoid duplicate attendance?

The service checks for an existing record, and the database has a UNIQUE constraint on (student, session), so even simultaneous frames cannot create two records.

Q: How does it handle many students at once?

YuNet detects all faces in one pass; each face is tracked, recognised and checked independently. About 10–15 faces per frame are processed within the 500 ms budget on a CPU.

Q: What if lighting is poor?

Accuracy drops in very dark conditions; the voting over several frames helps, and the teacher can override. The evaluation script can measure performance per lighting condition, which should be part of the final report.

Q: How is privacy protected?

Consent is required, no photos are stored, only encrypted embeddings; roles restrict access; face data can be deleted; the system only runs during teacher-started sessions.

Q: Why a PWA instead of a native Android app?

One code base for PC, Android and iPhone; installable without an app store; the AI runs on the server, so the phone only needs a camera and a browser. A Play Store version can be produced from the same PWA later.

Q: What are FAR and FRR?

FAR: how often a wrong person is accepted (for attendance, the dangerous error).

FRR: how often a registered person is not recognised (inconvenient but correctable).

Raising the threshold lowers FAR and raises FRR; we chose the point with FAR = 0 and FRR = 0 on our data.

Q: What are the main limitations?

Small evaluation dataset, anti-spoofing validated on few real attacks, need for adequate light and face size, CPU limits for very large classes, and possible demographic bias of pre-trained models.

Appendix A: Glossary

Term	Meaning
AI (Artificial Intelligence)	Computer systems that perform tasks normally requiring human intelligence, such as recognising faces.
CNN	Convolutional Neural Network: a deep-learning model for images.
Inference	Using a trained model on new data (as opposed to training it).
ONNX	Open Neural Network Exchange: a standard file format for trained networks.
Bounding box	Rectangle around a detected face (x, y, width, height).
Landmarks	Key points of a face: eyes, nose tip, mouth corners.
Alignment	Rotating and scaling a face so that its landmarks are at standard positions.
Embedding	Numerical vector (128 numbers) representing a face's identity.
Cosine similarity	Similarity of two vectors from their angle: 1 identical, 0 unrelated.
Threshold / margin	Minimum similarity to accept a match / minimum lead over the second-best candidate.
1:1 vs 1:N	Verifying one claimed identity vs identifying a person among N registered people.
IoU	Intersection over Union: overlap of two boxes divided by their combined area.
Liveness / anti-spoofing	Checking that a face is a live person, not a photo, screen or mask.
Presentation attack	Attempt to fool the system with a photo, video or mask.
FAR / FRR / EER	False Acceptance Rate / False Rejection Rate / Equal Error Rate where both are equal.
APCER / BPCER	Attacks accepted as live / genuine users rejected as attacks.
ROC AUC	Area under the ROC curve; 1.0 means perfect separation of same and different people.
ORM	Object-Relational Mapping: working with database tables as program objects (SQLAlchemy).
PWA	Progressive Web App: a website installable like an app, with offline support.
Service worker	Background browser script enabling offline behaviour and installation.
Certificate authority (CA)	Issuer of certificates that devices trust for HTTPS.
Fernet / PBKDF2	Authenticated encryption scheme / slow password-hashing function.

Appendix B: Configuration and API Reference

B.1 Main settings

Setting	Default	Effect
MATCH_THRESHOLD	0.45	Minimum cosine similarity for a match
MATCH_MARGIN	0.05	Lead required over the second-best student
VOTES_REQUIRED	3	Agreeing frames (of the last 5) before marking
DETECTION_SCORE / MIN_FACE_SIZE	0.85 / 40 px	Detector confidence and minimum face size
LIVENESS_MODE	auto	auto cnn motion cnn+motion
ANTISPOOF_THRESHOLD	0.7	Median CNN P(live) needed to accept (below 0.3 = spoof)
LIVENESS_MOTION_THRESHOLD	0.12	3-D motion required ($\approx 20^\circ$ head turn)
LIVENESS_MIN_FRAMES / TIMEOUT	6 / 12 s	Observation window of the head-movement test
Present / late window	10 / 20 min	Status rules (per session)
Low-attendance threshold	75%	Alert threshold (at least 3 classes)
SECRET_KEY, DATABASE_URL, SMTP_*	—	Security key, database, optional e-mail

B.2 Main endpoints

Method & path	Purpose
GET/POST /login, GET /logout	Authentication
GET /, GET /me	Dashboard (staff) / own attendance (student)
/students, /students/new, /students/{id}, /students/import	Student management and bulk import
POST /api/students/{id}/faces	Enrol face images (JSON with data URLs)
/courses, /courses/{id}	Courses and enrolments
/sessions, /sessions/new, /sessions/{id}, /sessions/{id}/live	Sessions, results, live camera page
POST /api/sessions/{id}/frame	Process one camera frame → faces, states, summary
POST /sessions/{id}/close, /sessions/{id}/override	Close session; manual status
/analytics, /api/analytics	Charts and statistics
/reports/daily, /reports/monthly, /reports/session/{id} (?fmt=html xlsx pdf csv)	Reports and exports
/users, /settings, /notifications, /audit	Administration
/mobile, /manifest.webmanifest, /sw.js	Mobile app setup and PWA files

End of notes.